Version control and collaboration with Git

Software Management & Development



IE University · MFT & MCSDI

Evaluation

Evaluation

Category	Weight
Participation	10 %
Individual work: five checkpoints of your own repository	40 %
Group work, with individual assessment	30 %
Final exam	20 %

> Participation is assessed by the professor. The individual work is separate from it.

Individual work: five checkpoints

Checkpoint	What your repository shows	Due
1 · Git	the full Git workflow, done by hand	Fri 13 Nov
2 · Structure and data	the app as a package, with a database	Fri 20 Nov
3 · API and tests	a REST API with tests that pass	Fri 27 Nov
4 · Frontend and containers	a web page using the API, Docker Compose	Fri 11 Dec
5 · Cloud	the app running on Azure	Fri 18 Dec

> Labs happen in class. You submit the link on Blackboard by Friday, 23:59.

Using AI

Allowed
under the course rules:
you answer for every line

→
unless

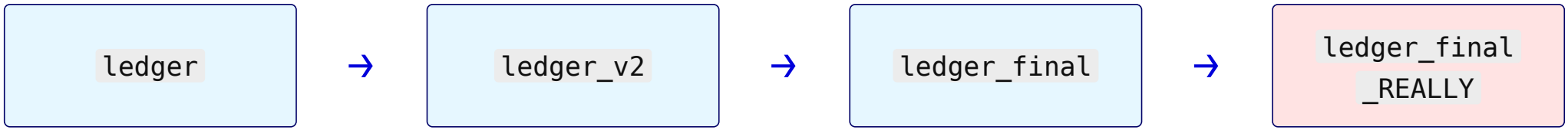
Not allowed
when an activity
says so explicitly

Today: **Git by hand**. Type every command and read what Git answers.

> Git is how you recover when an assistant breaks your project. Learn it first.

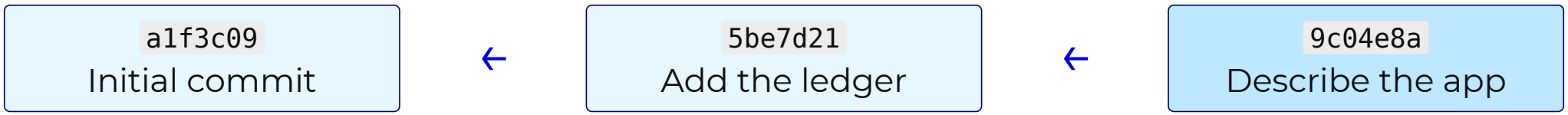
Version control

Without version control



- Which copy works? What changed between two of them, and why?
- Four people edit the same files: who overwrote whose work?

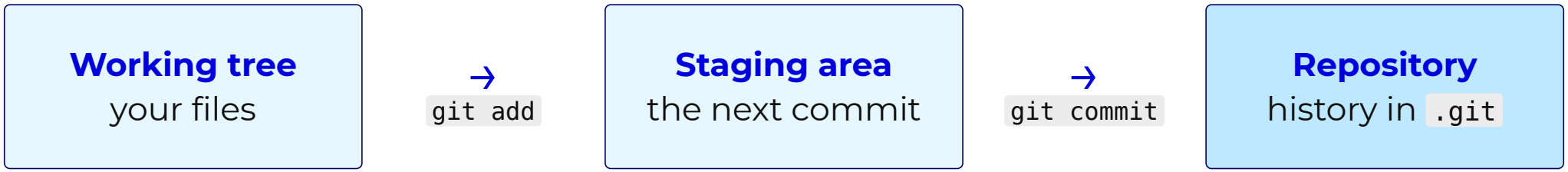
Commit



A snapshot of every tracked file, with author, date, message and a parent.

> The hash is computed from the contents and the parent: an old commit can't change without everyone noticing.

The three areas



`git status` shows where each change is. `git diff --staged` shows what you're about to record.

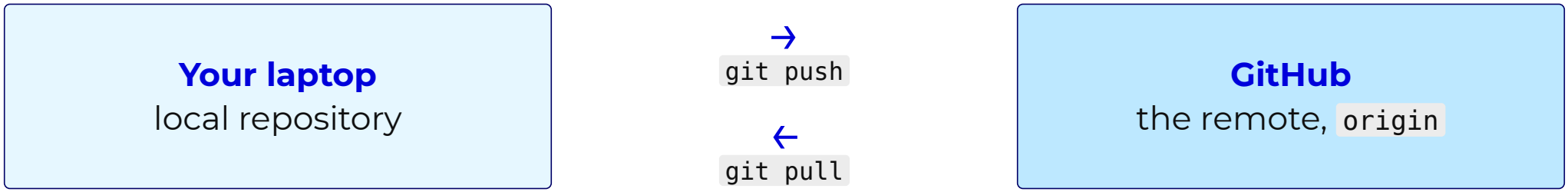
What goes in the repository

Commit	**Keep out (`.gitignore`)**
`pyproject.toml`, `uv.lock`, `.python-version`	`.venv/`: rebuilt by `uv`
`src/`, `README.md`	`__pycache__/`, `.ruff_cache/`
sample data the code needs	`.env` with passwords and keys

> A secret that was committed stays in the history. Revoke it and create a new one.

GitHub

Local and remote



- `git clone <url>`: copy a repository with its whole history
- Your commits reach GitHub only when you push; theirs reach you only when you pull

> `git pull` = `git fetch` (download) + `git merge` (combine).

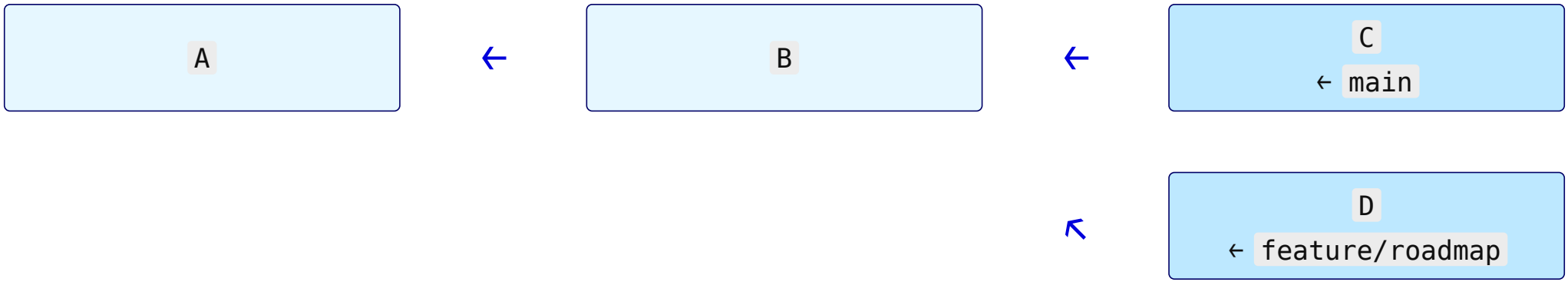
Signing in to GitHub

```
gh auth login        # GitHub.com → HTTPS → Yes → web browser
gh auth status
```

- The GitHub CLI signs you in once, through the browser
- Git uses that login for every push and pull: no passwords, no keys

Branches

Branch



A branch is a name that points to a commit, and moves forward when you commit on it.

> `git switch -c feature/roadmap` creates a branch and moves to it. `main` holds the official history.

Merge

Fast-forward
only one side has new commits:
`main` just moves forward

Merge commit
both sides have new commits:
a commit with two parents

Git combines changes on its own when they touch different lines.

Conflict

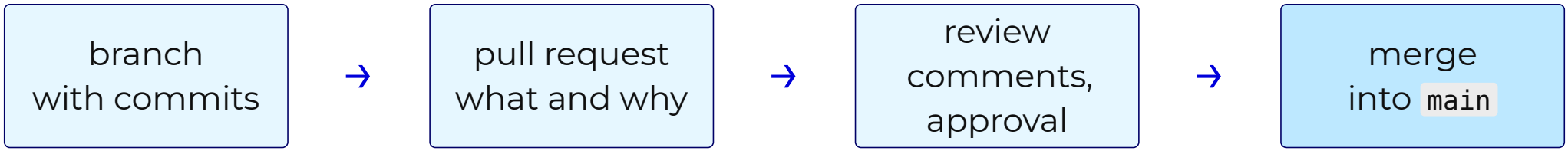
Both sides changed **the same line**. Git stops and shows both versions:

```
<<<<<<< HEAD
Personal finance ledger: balances, spending and monthly reports.
=======
Personal finance ledger for the command line.
>>>>>>> c3b83bd2a4f5...
```

- Edit the file so it says what it should, and delete the markers
- `git add` the file, then `git commit` to finish the merge

> A conflict is not an error: Git refuses to guess what the text means.

Pull request



At least two people have read every change before it reaches `main`.

> We merge with **Create a merge commit**: the history is part of what we assess.

Group project

One product, one module per member

member 1 → module

member 2 → module

...

member 6 → module

→ integrate

The team's product
one database, one API,
one web frontend,
Docker Compose, CI, Azure

> Your module is the application you build in your own repository, checkpoint by checkpoint.

Your module and your shared job

Module

- One main entity: create, read, update and delete through the API
- One calculated result: a balance, a report, a forecast
- Tests and one page in the frontend

Shared job, one per member

CI · Docker Compose and configuration · Azure deployment · frontend shell and dashboard · database · code review · documentation and board

Themes

Freelancer finance · Shared household · Small shop · Investment club · Micro-lending · SME back office · Trip planner · Crowdfunding · Insurance claims · Personal wealth

Example, **Freelancer finance**: clients · time tracking · invoices · payments received · expenses · tax estimator · cash-flow report

> One theme per team, first come, first served. You can propose your own if it splits into one module per member.

Group project: dates

Date	Milestone
Fri 20 Nov	team repository, theme, one module and one shared job per member
Mon 23 Nov	shared conventions agreed; first endpoint of each module
Fri 4 Dec	all modules in one product, through reviewed pull requests
Tue 15 Dec	running on Azure, built by GitHub Actions
Thu 17 Dec	demo and questions to every member

> Today, in your team: choose a theme and split it into modules.